can do to ameliorate the problem save for choosing a different division. In that case, choosing the appropriate division may require more knowledge of the target platform than you have available and is only worth doing if performance is unacceptable and it can be demonstrated that changing the division of work does improve performance.

Other factors can affect the performance of multithreaded code. The cost of cache ping-pong can vary quite considerably between two single-core processors and a single dual-core processor, even if they're the same CPU type and clock speed, for example, but these are the major ones that will have a very visible impact. Let's now look at how that affects the design of the code and data structures.

8.3. Designing data structures for multithreaded performance

In section 8.1 we looked at various ways of dividing work between threads, and in section 8.2 we looked at various factors that can affect the performance of your code. How can you use this information when designing data structures for multithreaded performance? This is a different question than that addressed in chapters 6 and 7, which were about designing data structures that are safe for concurrent access. As you've just seen in section 8.2, the layout of the data used by a single thread can have an impact, even if that data isn't shared with any other threads.

The key things to bear in mind when designing your data structures for multithreaded performance are *contention*, *false sharing*, and *data proximity*. All three of these can have a big impact on performance, and you can often improve things just by altering the data layout or changing which data elements are assigned to which thread. First off, let's look at an easy win: dividing array elements between threads.

8.3.1. Dividing array elements for complex operations

Suppose you're doing some heavy-duty math, and you need to multiply two large square matrices together. To multiply matrices, you multiply each element in the first *row* of the first matrix with the corresponding element